SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR

॥वसुधैव कुटुम्बकम्॥

COMPUTER SCIENCE AND ENGINEERING

BATCH 2023-2027

M I N I P R O J E C T R E P O R T

Real-Time Financial Fraud Detection Lakehouse Using Distributed Streaming and Machine Learning

A Big Data Approach Using Apache Kafka, PySpark Structured Streaming, Delta Lake and Random Forest

Submitted by:
Bhumika Raut – 23070521221
Arpita Mahakalkar – 23070521020

Domain: Big Data Analytics / Data Science
Guided by: Dr. Dharmaveer Chaudhary
Department: Computer Science and Engineering, Symbiosis Institute of Technology, Nagpur

Academic Year: 2026–27

Date: 10th October, 2026

Abstract

Digital payments generate transactions faster than analysts can inspect them, so fraud must be detected while the transaction stream is still flowing. This project presents a real-time financial fraud detection lakehouse built from Apache Kafka, PySpark Structured Streaming, Delta Lake, Spark MLlib, Docker and Streamlit. A synthetic generator publishes transactions (identifier, user, event time, amount, merchant category and device) to a six-partition Kafka topic keyed by user. A medallion pipeline preserves raw events in a Bronze layer, validates them in a Silver layer using eleven quality rules with a five-minute watermark for transaction-ID deduplication, and routes invalid events to a quarantine table with explicit rejection reasons. Two behavioural feature paths are computed: 10-minute sliding-window aggregates (2-minute slide) and a persistent per-user history maintained with applyInPandasWithState using Welford's online mean-and-variance algorithm. A weighted, interpretable rule engine and a Spark MLlib Random Forest (200 trees, depth 10, class-weighted, temporal 80/20 split) are combined by a hybrid decision engine that produces a fraud flag, severity, decision basis and human-readable reasons. The repository reports, on a temporal hold-out set of 1,753 transactions, ROC-AUC 0.9772, PR-AUC 0.7027, precision 0.6526, recall 0.6739, F1 0.6631 and accuracy 0.9641 (62 TP, 33 FP, 30 FN, 1,628 TN). A shuffle-partition benchmark under balanced and 90% hot-key traffic selected four partitions (up to 5,655 rows/s balanced, 7,469 rows/s skewed). Reliability is provided by checkpoints, Delta transactional writes, Docker restart policies and integration tests, and results are exposed through a Streamlit Fraud Command Center and the Spark History Server.

Keywords

Fraud Detection, Real-Time Analytics, Apache Kafka, PySpark Structured Streaming, Delta Lake, Medallion Architecture, Stateful Processing, Random Forest, Hybrid Rule-ML Decision.

Table of Contents

Section	Page
1. Introduction	**3**
1.1 Background and Context	3
1.2 Motivation	3
1.3 Problem Statement	3
1.4 Objectives of the Project	4
1.5 Scope and Evidence Basis	4
1.6 Novelty and Key Contributions	4
2. Literature Review / Related Work	**5**
2.1 Comparison of Techniques	6
2.2 Research Gaps and Proposed Solution	6
3. Requirements and System Architecture	**6**
3.1 Functional and Non-Functional Requirements	6
3.2 System Architecture	7
3.3 Technology Stack and Justification	8
4. Data and Methodology	**8**
4.1 Transaction Data and Data Contract	8
4.2 Data Quality, Deduplication and Quarantine	9
4.3 Bronze–Silver–Gold Lakehouse Design	9
4.4 Sliding-Window Features	9
4.5 Stateful Per-User Behavioural Features	10
4.6 Rule Engine	10
4.7 Random Forest Pipeline	10
4.8 Hybrid Final Decision	11
5. Reliability, Monitoring, Testing and Deployment	**11**
5.1 Fault Tolerance and Idempotency	11
5.2 Observability	11
5.3 Testing	12
5.4 Docker Deployment	12
6. Results and Discussion	**12**
6.1 Model Evaluation	12
6.2 Performance Benchmark	14
6.3 Dashboard and Gallery Outputs	15
6.4 Evidence Classification	17
7. Limitations and Future Scope	**18**
8. Conclusion	**18**
9. References	**19**

1. Introduction

1.1 Background and Context

Card payments, wallets and online banking now produce a continuous stream of events. Fraudulent activity hides inside this stream: an individual transaction may look normal in isolation yet be suspicious when compared with the same customer's spending history, the number of transactions in the last few minutes, or a sudden change of device. Detecting such patterns requires systems that process events as they occur, remember behaviour over long periods, and still produce decisions within seconds.

Batch analytics over yesterday's data cannot support this requirement, while a standalone machine-learning notebook cannot ingest, validate, deduplicate and score events at scale. Modern data platforms therefore combine a distributed log (Apache Kafka) for ingestion, a distributed stream-processing engine (Apache Spark Structured Streaming) for event-time computation, and a transactional storage format (Delta Lake) that gives streaming jobs reliable, queryable tables. Organising these tables into Bronze, Silver and Gold layers – the medallion or lakehouse pattern – separates raw evidence from validated data and from serving-ready outputs [3]–[5].

1.2 Motivation

Fraud is both stateful and time-sensitive. A purchase of 1,000 may be routine for one customer and extreme for another; five transactions within ten minutes may be normal at a shopping event and alarming for an account that normally transacts once a day. A useful system must therefore maintain per-user behavioural state and recent-activity windows, handle late and duplicate events, survive restarts without losing or double-counting data, and expose its decisions in a form analysts can inspect. From an academic perspective the project also integrates, in one reproducible stack, the main topics of a big-data course: distributed ingestion, event-time stream processing, storage design, feature engineering, machine learning, fault tolerance, benchmarking and monitoring.

1.3 Problem Statement

To design and implement an end-to-end, containerised, real-time fraud detection platform that ingests transaction events from Kafka, validates and deduplicates them, derives behavioural features from sliding windows and persistent per-user state, scores each transaction with a Random Forest model combined with an interpretable rule engine, and presents fraud alerts and engineering-health metrics on a monitoring dashboard – while remaining recoverable after failure and avoiding label leakage.

“How can a distributed streaming pipeline combine historical user behaviour, recent-window activity, machine learning and deterministic rules to flag fraudulent transactions reliably, explain why they were flagged, and recover safely from failure?”

1.4 Objectives of the Project

- Generate realistic synthetic transactions containing normal behaviour, fraud patterns, duplicates and late-arriving events, and publish them to Kafka without exposing fraud labels in the production payload.
- Persist raw events with Kafka lineage in a Bronze Delta table and validate them into a Silver table with explicit quarantine of invalid records.
- Use event-time processing with a 5-minute watermark for bounded deduplication and late-data handling.
- Compute 10-minute / 2-minute sliding-window behavioural features and persistent per-user statistics using applyInPandasWithState and Welford's algorithm.
- Train a class-weighted Spark MLlib Random Forest with a temporal train/test split and evaluate it with ROC-AUC, PR-AUC, precision, recall, F1 and a confusion matrix.
- Combine model probability and rule score into a hybrid, explainable final decision (flag, severity, decision basis, reasons).
- Provide reliability (checkpoints, idempotent Delta writes, restart policies), observability (StreamingQueryListener, Spark History Server) and a Streamlit dashboard.
- Benchmark Spark shuffle-partition settings under balanced and skewed traffic and select the configuration from measurements.

1.5 Scope and Evidence Basis

This report describes the implementation in the project repository (github.com/Imperator8000/real-time-fraud-detection). The data are **synthetic** and produced by the project's own generator; no real customer or bank data are used. All model metrics quoted are those **reported in the repository documentation**; the authors of this report did not independently re-run the model-evaluation job when preparing it, and the model, data and checkpoint folders are runtime artefacts that are not committed to the repository. Results are therefore presented as repository-reported offline evaluation on simulated data and must not be read as evidence of production performance on real payments. Section 6.4 classifies each piece of evidence.

1.6 Novelty and Key Contributions

Each component of this project (Kafka, Spark, Delta Lake, Random Forest) is established technology; the contribution lies in how they are combined and in several deliberate engineering decisions:

- **Label isolation:** simulator labels (is_fraud_simulated, fraud_type) never enter the Kafka payload; ground truth is stored separately and joined only for training.
- **Leakage-safe state update:** each transaction is compared with the user's state before that transaction updates the state.
- **Constant-size user state:** Welford's online algorithm keeps mean and variance without storing every past transaction.
- **Two complementary feature views** – overlapping 10-minute windows and long-lived per-user history – assembled into one immutable Gold feature event.
- **Hybrid, explainable decisions:** three alert paths (high-confidence ML, rule plus ML confirmation, critical rule score) with severity and reasons.

- **Hot-key-aware tuning:** shuffle partitions chosen by benchmark under 90% single-user skew, with a documented decision not to salt user_id because that would split one user's fraud history.
- **Reliability evidence:** checkpoint-continuation and Delta-idempotency integration tests, plus Spark History Server evidence.

2. Literature Review / Related Work

Table 1 summarises the technologies and ideas on which the project builds. Entries are drawn from peer-reviewed papers and official documentation; full citations are given in Section 9.

Table 1. Summary of related work and enabling technologies

Reference	Method / Contribution	Relevance to this project	Limitation
Kreps et al. (2011) [1]	Kafka: distributed log-based messaging for high-throughput event collection	Partitioned, replayable ingestion of transactions	A transport, not an analytics engine
Zaharia et al. (2016) [2]	Apache Spark: unified engine for batch, SQL, streaming and ML	Single engine for streaming, features and MLlib	Cluster tuning and overhead on small jobs
Armbrust et al. (2018) [3]	Structured Streaming: declarative event-time API with watermarks and exactly-once-style semantics	Windows, watermarks, stateful operators	Late-data policy must be chosen by the user
Armbrust et al. (2020) [4]	Delta Lake: ACID tables over object stores with a transaction log	Reliable Bronze/Silver/Gold tables, idempotent writes	Requires Spark/Delta runtime
Armbrust et al. (2021) [5]	Lakehouse architecture unifying warehouse and lake	Medallion layering of data	Architectural pattern, not a fraud method
Akidau et al. (2015) [6]	Dataflow model: event time vs processing time, windows, watermarks	Conceptual basis for event-time design	General model
Meng et al. (2016) [7]	MLlib: scalable machine learning in Spark	Pipeline, Random Forest, imputation, encoding	Fewer algorithms than single-node libraries
Breiman (2001) [8]	Random Forests: ensembles of randomised trees	Core classifier; robust to non-linear tabular features	Probabilities need calibration; less transparent than a single tree
Welford (1962) [9]	Online algorithm for mean and variance	Constant-size per-user statistics	Assumes sequential updates
Bahnsen et al. (2016) [10]	Feature engineering for card fraud: aggregated and periodic features	Motivates velocity and behaviour-deviation features	Evaluated on batch data
Dal Pozzolo et al. (2018) [11]	Realistic fraud-detection modelling: delayed labels, concept drift, precision-oriented metrics	Motivates temporal evaluation and caution about accuracy	Proprietary card data
Saito & Rehmsmeier (2015) [12]	Precision-recall plots are more informative than ROC on imbalanced data	Justifies reporting PR-AUC with ROC-AUC	General binary classification

2.1 Comparison of Techniques

Rule-only systems are transparent and fast but brittle: thresholds must be tuned manually and sophisticated patterns evade them. Machine-learning-only systems learn non-linear interactions but give probabilities that are harder to justify to analysts and depend on the quality of historical labels [11]. Hybrid systems combine both: rules encode domain knowledge and provide reasons, while the model captures interactions among features. Tree ensembles such as Random Forest [8] are well suited to tabular behavioural features, cope with mixed scales and non-linear effects, and are available as a distributed implementation in Spark MLlib [7].

On the platform side, Kafka [1] separates producers from consumers and permits replay; Structured Streaming [3] offers event-time windows and watermarks; Delta Lake [4] adds transactional guarantees to the files that streaming jobs read and write; and the medallion layering [5] keeps raw data, validated data and serving data distinct. Stateful fraud features such as “average spend of this user so far” require per-key state, for which Structured Streaming provides arbitrary stateful processing; Welford's algorithm [9] keeps that state small.

2.2 Research Gaps and Proposed Solution

Many published and tutorial fraud systems are static notebooks trained on shuffled, already-labelled datasets. They rarely show (i) the ingestion and data-quality layers, (ii) both windowed and long-term behaviour computed in a stream, (iii) leakage-safe state updates and label isolation, (iv) recovery after failure, or (v) measured Spark tuning under skewed traffic. The proposed solution addresses these together in one containerised pipeline. We state the contribution conservatively: it is an integrated engineering implementation evaluated on synthetic data, not a new learning algorithm.

3. Requirements and System Architecture

3.1 Functional and Non-Functional Requirements

Table 2. Requirements

Type	Requirement
Functional	Ingest JSON transactions from Kafka; preserve raw record with topic/partition/offset
Functional	Validate records, deduplicate by transaction ID, quarantine invalid records with reasons
Functional	Compute sliding-window and persistent per-user behavioural features
Functional	Score each transaction with a Random Forest and rule engine; emit fraud flag, severity, basis and reasons
Functional	Store scored transactions and alerts; display them on a dashboard
Non-functional	Recover from process failure without reprocessing from scratch or duplicating output
Non-functional	Bounded state (watermark, state timeout, constant-size user state)
Non-functional	Reproducible deployment with Docker Compose; observable via Spark metrics
Non-functional	No label leakage into production payload or serving features

3.2 System Architecture

Figure 1 shows the complete architecture. Transactions flow from the generator to Kafka, through the Bronze, Silver and Gold layers of Delta Lake, into ML inference and the hybrid decision engine, and finally to the dashboard. Reliability and observability mechanisms (checkpoints, Delta transaction identifiers, StreamingQueryListener, Spark History Server) act across the streaming applications. Each Spark stage runs as a separate application/container, so a failure in one is isolated and independently restartable.

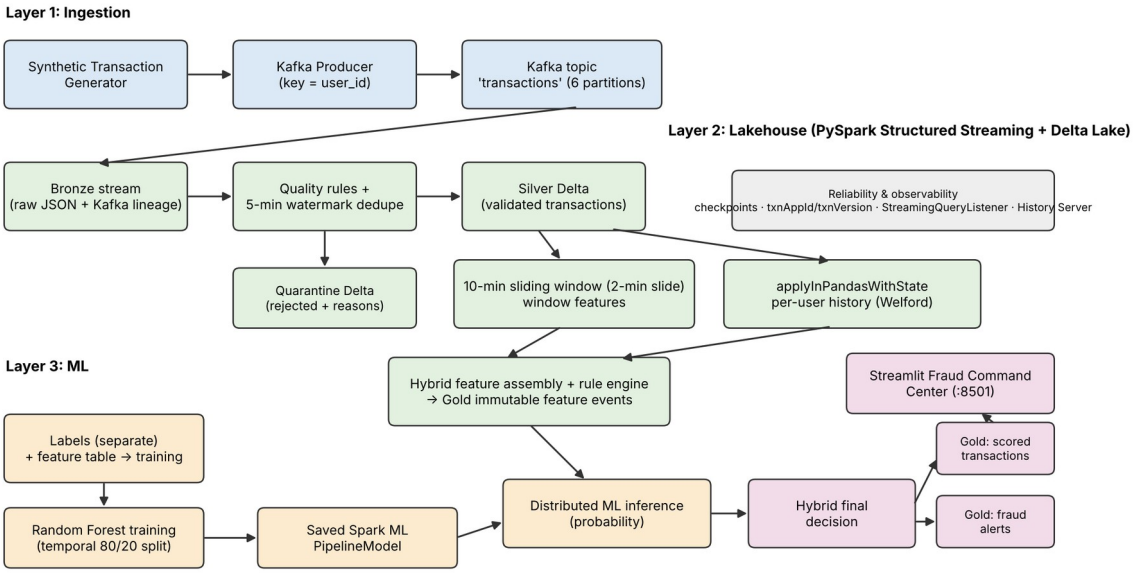


Figure 1. System architecture of the Real-Time Financial Fraud Detection Lakehouse (drawn from repository code and docs/architecture.md)

End-to-end flow of one transaction. (1) The generator creates a transaction and the producer sends it to the Kafka topic using user_id as key, so one user's events go to one partition. (2) The Bronze stream stores the raw JSON with Kafka lineage. (3) The quality stage parses the JSON and applies eleven validation rules; invalid events go to quarantine, valid ones are deduplicated within the 5-minute watermark and written to Silver. (4) Silver feeds two stateful paths: the sliding-window aggregation and the per-user state function. (5) The assembly stage joins the transaction's historical features with the applicable window features, evaluates the rule engine and writes an immutable Gold feature event. (6) The scoring stage loads the saved PipelineModel, adds the Random Forest probability, applies the hybrid decision and writes scored transactions and fraud alerts. (7) The dashboard reads committed Gold snapshots.

3.3 Technology Stack and Justification

Table 3. Technology stack (versions as configured in the repository where stated)

Component	Role in the project	Why it is used
Apache Kafka 4.3.1 (Docker image)	Event transport; topic “transactions”, 6 partitions, key = user_id	Decouples producers from consumers; replay; per-user locality
PySpark / Spark 3.5.9	Structured Streaming, windows, stateful processing, MLlib	Distributed event-time and stateful processing in Python
Delta Lake	Bronze, Silver, Gold, quarantine tables; MERGE; txnAppId/txnVersion	ACID commits, streaming sinks, idempotent writes
Spark MLlib	Imputer, StringIndexer, OneHotEncoder, VectorAssembler, RandomForestClassifier	Distributed training and inference in one persisted PipelineModel
Pandas / NumPy	Per-group logic inside applyInPandasWithState	Readable per-user state update
Streamlit	Fraud Command Center dashboard (port 8501)	Lightweight Python UI, separate container
Spark History Server	Post-run job, stage and executor inspection (port 18080)	Spark-level observability
Docker / Docker Compose	Orchestrates broker, streams, history server, dashboard	Reproducible multi-service deployment, restart policies
pytest	Unit and integration tests	Automated verification of key logic

4. Data and Methodology

4.1 Transaction Data and Data Contract

The data source is a configurable synthetic generator (src/producer/transaction_generator.py). It simulates per-user behavioural profiles and merchant categories with category-specific amount multipliers, and deliberately injects fraud patterns, duplicate messages and late events. Default simulation rates in the generator are: fraud 2%, duplicates 1%, late-within-watermark 3% and late-beyond-watermark 1%. Fraud types in the generator include, among others, merchant anomaly patterns. The produced event contains only fields that would be available at processing time (Table 4); the labels is_fraud_simulated, fraud_type, lateness and duplicate metadata are held in a separate ground-truth dataset keyed by transaction_id.

Table 4. Kafka transaction event (data contract)

Field	Type	Description
transaction_id	string	Unique transaction identifier
user_id	string	Customer identifier (also Kafka key)
timestamp	timestamp	Event-time of the transaction
amount	double	Transaction amount
merchant_category	string	Merchant category
device_id	string	Device used

The final scored Gold record contains the original fields plus engineered features (for example historical_avg_amount, amount_deviation_ratio, amount_zscore, transactions_last_10m, spend_last_10m, transactions_per_minute, unique_devices_10m, new_device_flag), the rule outputs (fraud_rule_score, rule_anomaly_flag), the model outputs (ml_prediction, ml_fraud_probability) and the final decision (final_fraud_flag, final_fraud_severity, final_decision_basis, fraud_reasons), partitioned by event_date.

4.2 Data Quality, Deduplication and Quarantine

The quality module (src/quality/transaction_quality.py) adds a rejection-reason array to each Bronze record. One event can break several rules and **all** failures are recorded. The rules are: INVALID_JSON, MISSING_TRANSACTION_ID, MISSING_USER_ID, MISSING_EVENT_TIMESTAMP, MISSING_AMOUNT, NAN_AMOUNT, NON_POSITIVE_AMOUNT, EXCESSIVE_AMOUNT (above 1,000,000), MISSING_MERCHANT_CATEGORY, MISSING_DEVICE_ID and EVENT_TIMESTAMP_TOO_FAR_IN_FUTURE (more than 5 minutes ahead). Importantly, **suspicious-looking but structurally valid** transactions (for example a very large purchase below the limit) are not rejected: they continue to the fraud engine. Invalid records are written to a quarantine Delta table with Kafka lineage and the original JSON so each can be traced to its source offset. Valid records are deduplicated using a 5-minute event-time watermark and dropDuplicatesWithinWatermark on the transaction ID, which bounds the memory needed to remember seen IDs.

4.3 Bronze–Silver–Gold Lakehouse Design

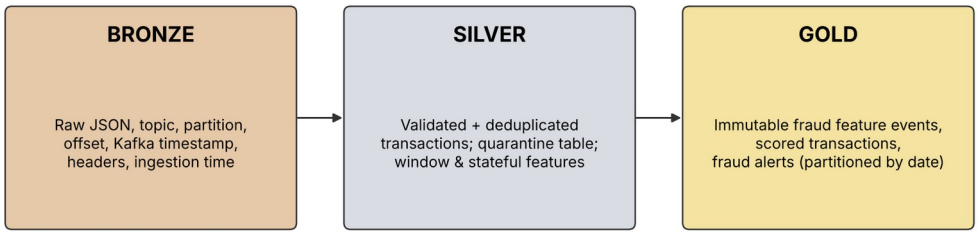


Figure 2. Medallion (Bronze–Silver–Gold) data layers used in the project

- **Bronze** preserves the Kafka record unchanged (raw JSON, topic, partition, offset, Kafka timestamp, headers, ingestion timestamp). It supports replay and audit. The Bronze stream reads Kafka from the earliest offset with a bounded maximum of offsets per trigger (default 5,000) and a 5-second trigger.
- **Silver** holds validated, deduplicated transactions, the quarantine table, the sliding-window features and the stateful per-user features.
- **Gold** holds immutable, append-only fraud feature events and the final serving tables (scored transactions and fraud alerts), partitioned by event date. The design notes that MERGE-maintained training snapshots are kept separate from immutable serving events, because a MERGE-updated table is not a normal append-only streaming source.

4.4 Sliding-Window Features

Silver transactions are aggregated per user over 10-minute event-time windows that slide every 2 minutes, with a 5-minute watermark. Because windows overlap, one transaction contributes to several windows. Features include transaction count, total and average spending, maximum, minimum and standard deviation of amount, transactions per minute, spending per minute, unique devices and unique merchant categories, and derived ratios such as device-diversity ratio and merchant-diversity ratio. These capture short-term velocity and diversity that a single transaction cannot reveal.

4.5 Stateful Per-User Behavioural Features

A second path uses applyInPandasWithState to keep one persistent state record per user: historical transaction count, running mean, variance state (M2), last event time, previous device and device-change count. State is updated with Welford's online algorithm [9]:

$$n \leftarrow n + 1;\ \delta \leftarrow x - \mu;\ \mu \leftarrow \mu + \delta / n;\ M2 \leftarrow M2 + \delta\,(x - \mu);\ s^2 = M2 / (n - 1)$$

so mean and sample variance are obtained in O(1) space. For every transaction the pipeline first computes the historical features from the state **before** the update – historical mean, standard deviation, amount-to-history ratio, z-score, seconds since the previous transaction and device change – and only then folds the current amount into the state. This prevents the transaction from influencing its own baseline. Per-user state expires after a configurable inactivity timeout (default 30 minutes in the code), which bounds state-store size. The unit tests verify the Welford statistics.

4.6 Rule Engine

The rule engine (src/features/fraud_feature_assembly.py) evaluates eight signals and adds their weights to form an interpretable fraud_rule_score. The rule flag is raised when the score reaches the configured threshold (default 4).

Table 5. Rule signals, weights and default thresholds (from the source code)

Rule	Weight	Default threshold / meaning
rule_historical_amount_anomaly	3	Amount anomalous relative to user history
rule_extreme_zscore	2	z-score ≥ 3.0
rule_high_amount_ratio	2	Amount ÷ historical mean ≥ 3.0
rule_high_velocity	2	≥ 5 transactions in the 10-minute window
rule_multi_device_activity	1	Multiple devices in the window
rule_device_change_high_amount	2	Device changed and amount ratio ≥ 2.0
rule_high_merchant_diversity	1	Merchant-diversity ratio ≥ 0.80
rule_rapid_repeat	1	Rapid repeat transactions

Thresholds are read from environment variables, so they can be tuned without code changes. The maximum possible rule score is 14.

4.7 Random Forest Pipeline

Training (src/ml/train_random_forest.py) joins the engineered features with the separate ground-truth labels by transaction_id. The Spark ML pipeline contains: conversion of boolean flags to numeric; a median Imputer over 20 numeric features (amount, historical count/mean/stddev, amount ratio, z-score, seconds since previous transaction, device-change count, and the window features such as transaction_count_10m, total_spending_10m, transactions_per_minute, unique_devices_10m, merchant_diversity_ratio, etc.); StringIndexer and OneHotEncoder for merchant category; a VectorAssembler; and a RandomForestClassifier with numTrees = 200, maxDepth = 10, subsamplingRate = 0.8 and seed = 42. Class imbalance is handled by a class-weight column (inverse-frequency style weights computed from the training split). The split is **temporal**: events up to the 80th percentile of event time are used for training and later events for testing, so the model is never tested on data earlier than its training data. The whole PipelineModel is persisted and loaded once by the live scoring stream.

4.8 Hybrid Final Decision

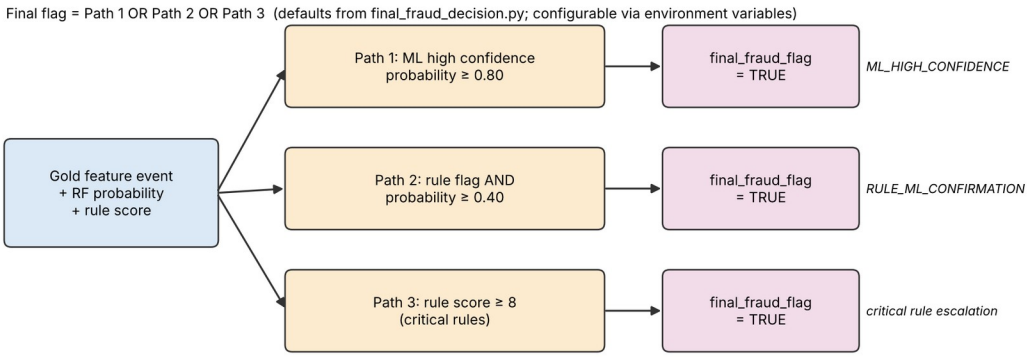


Figure 3. Hybrid final decision paths (defaults from src/features/final_fraud_decision.py)

A transaction is flagged when any of three paths holds: (1) the Random Forest probability is at least 0.80 (ML_HIGH_CONFIDENCE); (2) the rule flag is raised and the probability is at least 0.40 (RULE_ML_CONFIRMATION); or (3) the rule score is at least 8 (critical rules). Severity levels are LOW, MEDIUM, HIGH and CRITICAL, using additional thresholds (medium-risk 0.35; critical probability 0.95). A fraud_reasons array lists the active rule signals, making each alert explainable. All thresholds are environment-configurable.

5. Reliability, Monitoring, Testing and Deployment

5.1 Fault Tolerance and Idempotency

- **Kafka replay and Spark checkpoints:** each streaming query has its own checkpoint holding source offsets, committed micro-batches and state-store data, so a restart resumes where it stopped and restores per-user state.
- **Delta ACID commits** guarantee that each micro-batch output is atomic.
- **Idempotent writes:** foreachBatch sinks use Delta txnAppId and txnVersion (the Spark batch ID), so retrying the same batch does not add a duplicate logical commit.
- **Docker restart policy (unless-stopped)** restarts failed processes; Spark checkpoints restore progress. The two address different problems.
- **Quarantine** prevents silent loss of invalid events.

The repository notes that state schema and query semantics must not be changed casually while reusing a checkpoint.

5.2 Observability

A StreamingQueryListener records structured progress metrics (input rows, processing rate, trigger duration, planning time, state rows, state-store memory, watermark drops, source offsets) as JSONL. Spark event logs are persisted for the Spark History Server (port 18080), exposing jobs, stages, executors and SQL plans (Figures 12 and 13). The Streamlit dashboard reads committed Gold snapshots and does not itself score transactions.

5.3 Testing

The repository contains seven test functions in five files: unit tests for Welford statistics (test_welford.py), transaction-quality rules (test_transaction_quality.py) and the final fraud decision (test_final_fraud_decision.py), and integration tests for checkpoint recovery (test_checkpoint_recovery.py, which stops and restarts a file-source stream with the same checkpoint and verifies continuation without duplicating output) and Delta idempotency (test_delta_idempotency.py, which repeats a write with the same application ID and version). The tests are run with python -m pytest tests -v inside the Spark container. **Note:** test execution results are not stored in the repository and were not re-run for this report, so no pass/fail claim is made here.

5.4 Docker Deployment

docker/docker-compose.yml defines a Kafka broker (apache/kafka:4.3.1) with a topic-initialisation job (topic “transactions”, 6 partitions), the Spark History Server, one container per streaming application (bronze, silver, window, stateful, live-feature and final-scoring streams), a separate dashboard container (fraud-detection-dashboard:1.0, port 8501) and a development container. Streaming services use health checks and restart: unless-stopped. The dashboard is separated so that visualisation dependencies are not installed in every Spark image.

6. Results and Discussion

6.1 Model Evaluation

The repository README reports the following results from the project's generated model-evaluation output, using the temporal hold-out split described in Section 4.7. They are quoted here as **repository-reported** values.

Table 6. Repository-reported evaluation on the temporal hold-out set

Metric	Value	Metric	Value
ROC-AUC	0.9772	True positives	62
PR-AUC	0.7027	False positives	33
Precision	0.6526	False negatives	30
Recall	0.6739	True negatives	1,628
F1-score	0.6631	Accuracy	0.9641

Consistency check. The confusion-matrix counts are internally consistent with the reported scores: the hold-out set contains 62 + 33 + 30 + 1,628 = 1,753 transactions, of which 92 are fraud (5.25%). Precision = 62/95 = 0.6526, recall = 62/92 = 0.6739, F1 = 0.6631 and accuracy = 1,690/1,753 = 0.9641, all matching the reported values. ROC-AUC and PR-AUC cannot be recomputed from the confusion matrix and are taken as reported.

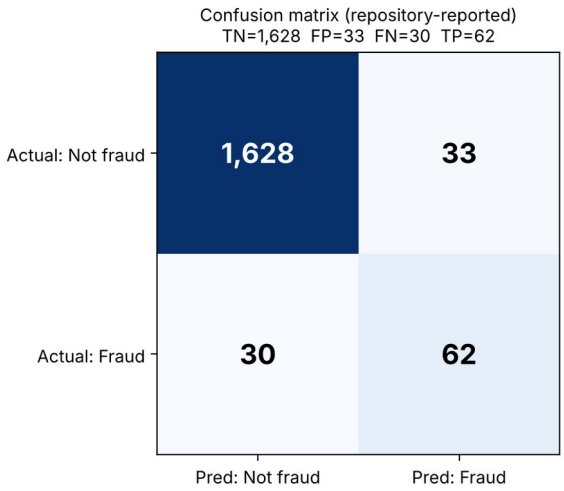


Figure 4. Confusion matrix drawn from the repository-reported counts

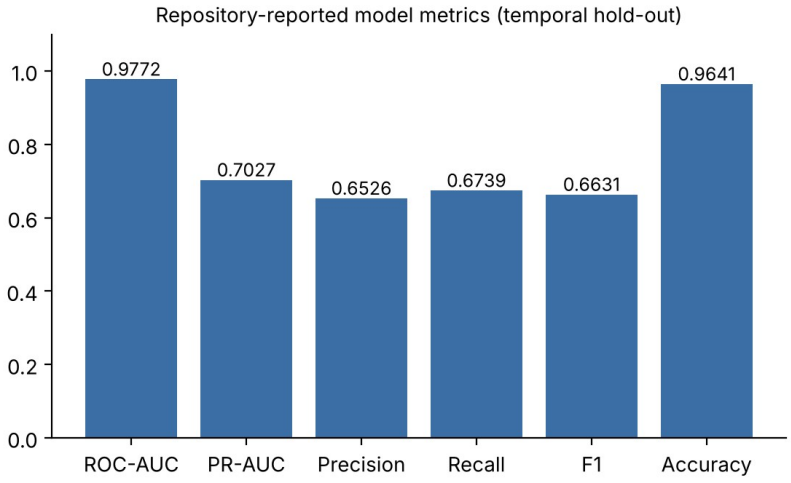


Figure 5. Repository-reported evaluation metrics

Why accuracy alone is misleading. Only about 5% of the hold-out transactions are fraud. A trivial classifier that labels every transaction "not fraud" would reach 1,661/1,753 = 94.75% accuracy while detecting no fraud at all; the model's 96.41% is therefore only modestly above this baseline and says little about fraud detection. Precision, recall, F1 and PR-AUC, which focus on the fraud class, are more informative [12]. The high ROC-AUC (0.9772) shows good ranking ability, whereas the lower PR-AUC (0.7027) reflects the difficulty of keeping precision high at high recall when positives are rare.

Precision–recall trade-off. At the reported operating point about two-thirds of alerts are genuine (precision 0.65) and about two-thirds of frauds are caught (recall 0.67). The 30 false negatives are missed frauds, each a direct financial loss; the 33 false positives are legitimate customers who may be inconvenienced or have cards blocked. Lowering the probability threshold would raise recall at the cost of more false positives, and vice versa; the right balance depends on the relative cost of fraud loss and customer friction. The hybrid design lets analysts adjust thresholds through configuration.

Limits of these results. The data are synthetic and generated by the same simulator whose patterns the rules and features target, so performance on real transactions may differ substantially. The hold-out size (92 fraud cases) is small, so the metrics have wide

uncertainty; no confidence intervals or repeated runs are reported. These offline metrics do not demonstrate live production performance.

6.2 Performance Benchmark

The benchmark (src/monitoring/benchmark_streaming_state.py) measured spark.sql.shuffle.partitions of 4, 8, 16 and 32 with the same 10-minute window, 2-minute slide, 5-minute watermark and 5-second trigger, under balanced traffic and under a skewed workload in which 90% of records belong to one user. It was run on a single-node local setup, so absolute figures are not cluster results.

Table 7. Repository-reported benchmark results

Workload	Partitions	Rows/s	P95 trigger (ms)	Max state memory (MB)
balanced	4	5,655.23	1,066	10.36
balanced	8	5,247.84	1,150	10.14
balanced	16	4,294.62	1,315	10.74
balanced	32	3,340.28	1,637	13.14
skewed	4	7,468.76	893	1.09
skewed	8	6,449.53	829	1.14
skewed	16	5,154.06	1,092	1.20
skewed	32	3,361.93	1,723	1.34

All configurations met the 5-second trigger budget. Four partitions gave the highest throughput in both workloads and was selected (baseline 8): throughput +7.76% (balanced) and +15.80% (skewed); P95 latency improved 7.30% in the balanced case but was 7.72% worse in the skewed case, where 8 partitions had the lowest P95 (829 ms versus 893 ms). More partitions did not help skewed traffic because a single hot user is one logical state key; the project deliberately does not salt user_id, since salting would split a user's fraud history across independent states.

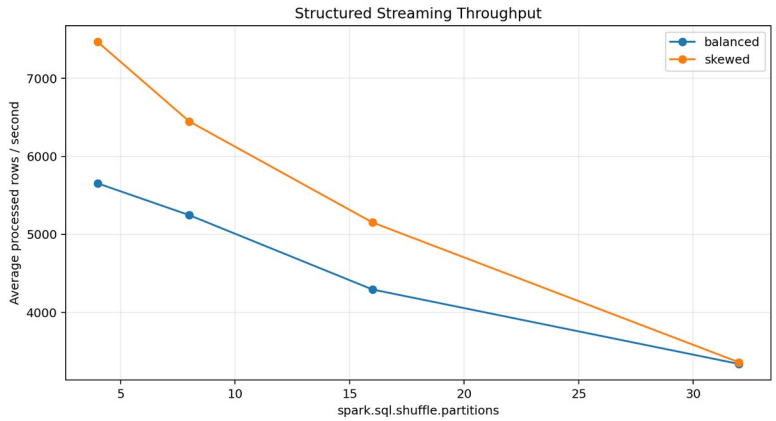


Figure 6. Processing throughput by shuffle-partition count (docs/performance)

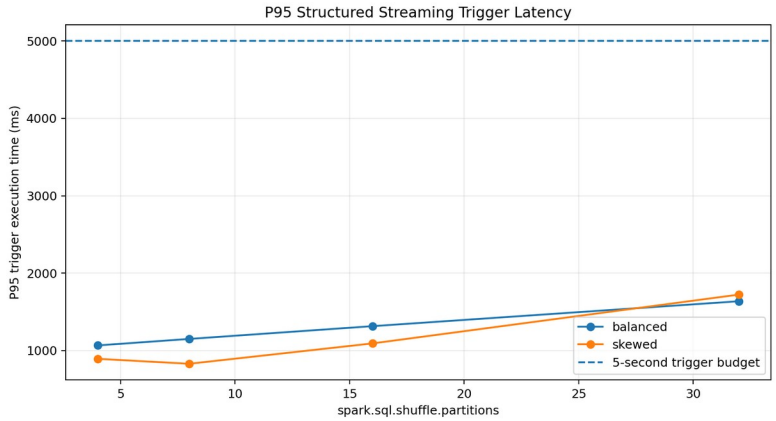


Figure 7. P95 trigger latency by shuffle-partition count (docs/performance)

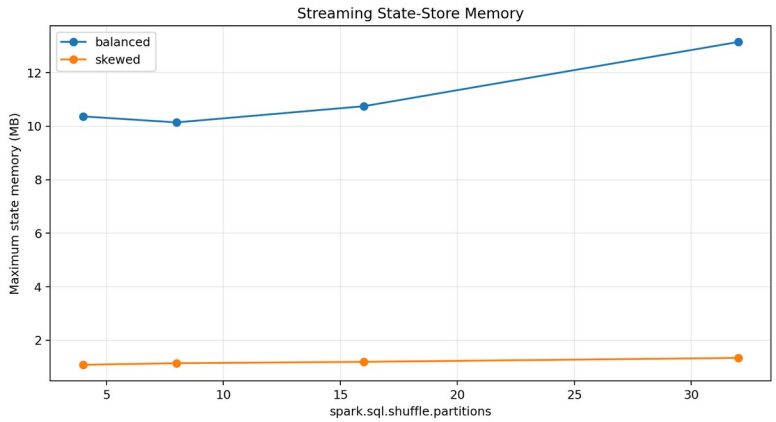


Figure 8. State-store memory by shuffle-partition count (docs/performance)

6.3 Dashboard and Gallery Outputs

The following figures are genuine screenshots stored in the repository (docs/screenshots). They come from a demonstration run of the platform on simulated data and show the committed Gold outputs and Spark execution evidence.

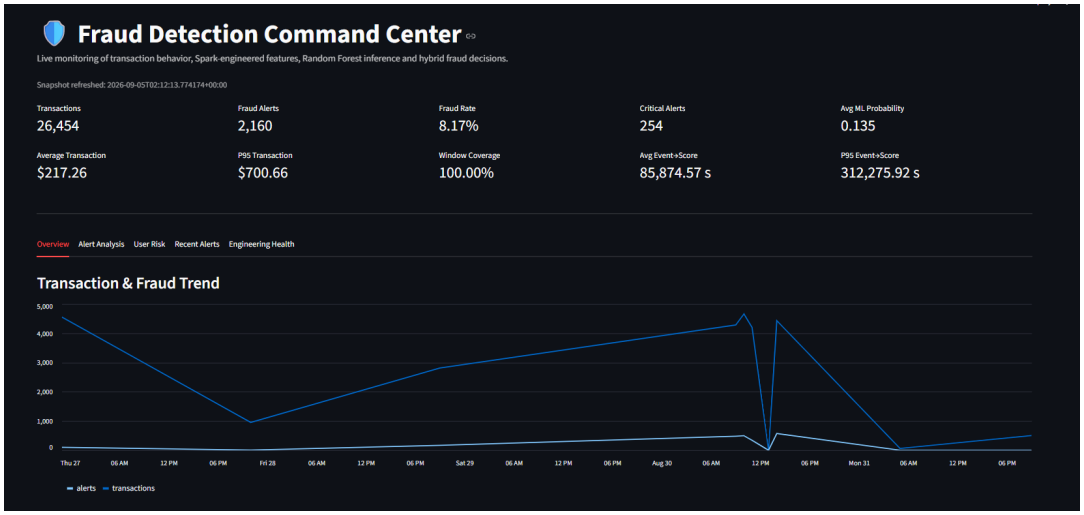


Figure 9. Fraud Detection Command Center – overview (snapshot 2026-09-05). KPIs: 26,454 transactions, 2,160 fraud alerts (8.17%), 254 critical alerts, average ML probability 0.135, average transaction $217.26, P95 transaction $700.66, window coverage 100%.

The overview shows transaction and alert counts over time, with tabs for Alert Analysis, User Risk, Recent Alerts and Engineering Health. Note that the alert rate of 8.17% in this demonstration is higher than the 2% simulated fraud rate and than the 5.25% fraud share of the evaluation set; it counts all flagged transactions, including false positives from the rule and hybrid paths, in a replayed demonstration stream. It should not be read as the model's precision.

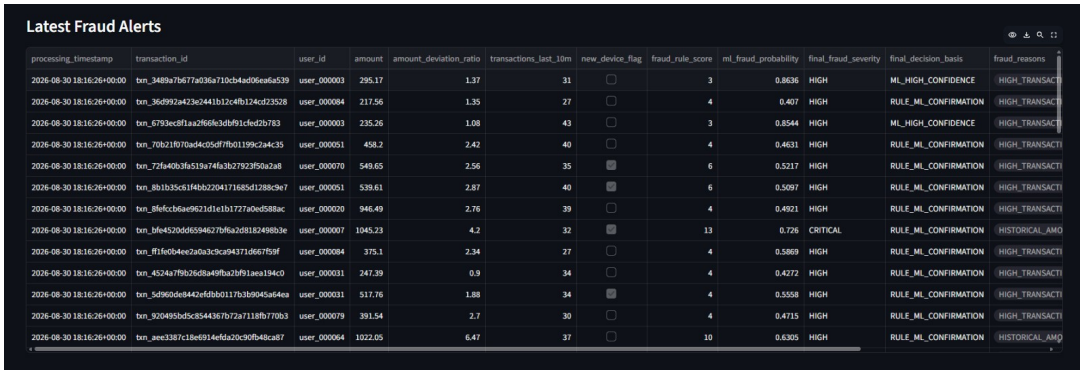


Figure 10. Latest Fraud Alerts table with amount, deviation ratio, 10-minute transaction count, device flag, rule score, ML probability, severity, decision basis and reasons

Each alert is explainable. For example, the highlighted CRITICAL alert (user_000007, amount 1,045.23, deviation ratio 4.2, rule score 13, ML probability 0.726) was raised on the RULE_ML_CONFIRMATION path, while other alerts with probability 0.86 used ML_HIGH_CONFIDENCE.

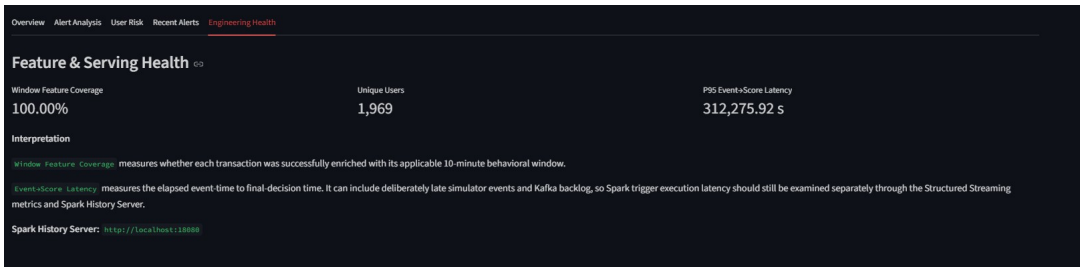


Figure 11. Engineering Health tab: window-feature coverage 100%, 1,969 unique users, P95 event-to-score latency

The dashboard itself cautions that event-to-score latency is measured from event time to decision time and “can include deliberately late simulator events and Kafka backlog”; the very large values displayed (average 85,874.57 s; P95 312,275.92 s) therefore reflect replayed or late events and backlog in the demonstration, not the Spark trigger latency, which is measured separately (Table 7).

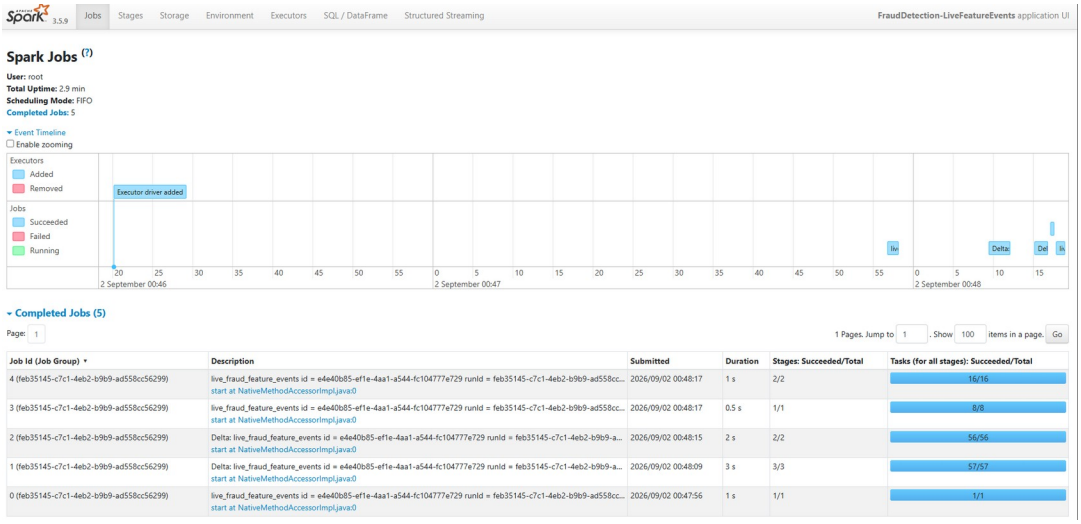


Figure 12. Spark History Server – Jobs for the FraudDetection-LiveFeatureEvents application (5 completed jobs, all stages succeeded)

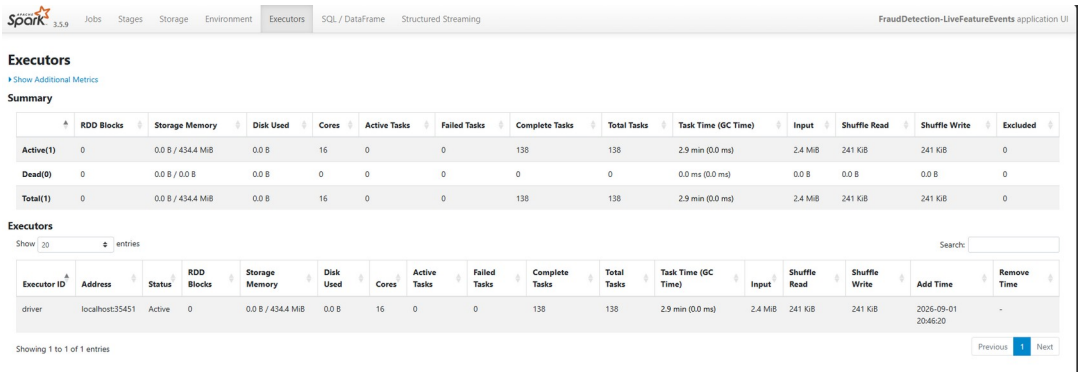


Figure 13. Spark History Server – Executors (1 active driver executor, 16 cores, 138 complete tasks, 0 failed tasks)

6.4 Evidence Classification

Table 8. What each piece of evidence demonstrates

Evidence	Type	What it supports	What it does not show
Table 6 metrics	Repository-reported offline evaluation (synthetic, temporal hold-out)	Model ranking and trade-off on simulated data	Real-world or live performance
Table 7 / Figs 6–8	Repository-reported local performance benchmark	Relative effect of shuffle partitions on one machine	Cluster scalability
Figs 9–11	Screenshots of a demonstration run (replayed simulated stream)	Dashboard features and explainable alerts	Precision or latency in production
Figs 12–13	Spark History Server screenshots	Spark jobs executed without failed tasks	Overall system correctness
Unit/integration tests	Test code in repository (results not re-run here)	Intended verification of key logic	Pass/fail status

7. Limitations and Future Scope

Limitations

- Data are synthetic and generated by a simulator aligned with the implemented rules; real payment data would contain concept drift, delayed labels and richer fraud tactics.
- The hold-out set contains only 92 fraud cases; no cross-validation, confidence intervals or comparison with other classifiers is reported.
- Precision (0.65) and recall (0.67) mean substantial false positives and missed frauds; probabilities are not calibrated.
- Rule weights and thresholds are hand-set defaults, not learned or independently validated.
- Benchmarks were run locally on one node; behaviour on a multi-node cluster and with a hot key at larger scale is untested.
- Event-to-score latency in the demonstration is dominated by replayed and late events, so end-to-end real-time latency is not established by the screenshots.
- The model, data and checkpoint folders are runtime artefacts not stored in the repository, so metrics cannot be reproduced without re-running training.

Future Scope

- Evaluate on public or partner transaction datasets and add cost-sensitive threshold selection.
- Compare Random Forest with gradient-boosted trees and calibrate probabilities; add feature-importance reporting.
- Add drift monitoring and scheduled retraining with delayed-label feedback.
- Run on a multi-node cluster and test larger hot-key scenarios; add alert delivery (e.g., notifications) and an analyst feedback loop.
- Add authentication and access control to the dashboard and encrypt sensitive fields for production use.

8. Conclusion

The project delivers an integrated real-time fraud detection platform in which Kafka, PySpark Structured Streaming and Delta Lake form a Bronze–Silver–Gold pipeline; sliding-window and Welford-based per-user state supply behavioural features; and a class-weighted Random Forest is combined with an interpretable rule engine into explainable hybrid decisions. The repository-reported evaluation on a temporal hold-out of simulated data (ROC-AUC 0.9772, PR-AUC 0.7027, precision 0.6526, recall 0.6739) shows useful but imperfect detection, and the analysis highlights why accuracy alone is inadequate on imbalanced data. Reliability is addressed through checkpoints, idempotent Delta writes, quarantine, Docker restart policies and integration tests, and performance was tuned by a measured benchmark that selected four shuffle partitions. The principal caveat is that all results come from synthetic data and a local deployment; validation on real transactions and a cluster remains future work.

9. References

[1] J. Kreps, N. Narkhede and J. Rao, “Kafka: a distributed messaging system for log processing,” in Proc. NetDB Workshop, 2011. Apache Kafka documentation: https://kafka.apache.org/documentation/

[2] M. Zaharia et al., “Apache Spark: a unified engine for big data processing,” Communications of the ACM, vol. 59, no. 11, pp. 56–65, 2016. doi:10.1145/2934664

[3] M. Armbrust et al., “Structured Streaming: a declarative API for real-time applications in Apache Spark,” in Proc. ACM SIGMOD, 2018. doi:10.1145/3183713.3190664

[4] M. Armbrust et al., “Delta Lake: high-performance ACID table storage over cloud object stores,” Proc. VLDB Endowment, vol. 13, no. 12, 2020. doi:10.14778/3415478.3415560

[5] M. Armbrust et al., “Lakehouse: a new generation of open platforms that unify data warehousing and advanced analytics,” in Proc. CIDR, 2021. http://cidrdb.org/cidr2021/papers/cidr2021_paper17.pdf

[6] T. Akidau et al., “The Dataflow Model: a practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing,” Proc. VLDB Endowment, vol. 8, no. 12, 2015. doi:10.14778/2824032.2824076

[7] X. Meng et al., “MLlib: machine learning in Apache Spark,” Journal of Machine Learning Research, vol. 17, no. 34, 2016. https://jmlr.org/papers/v17/15-237.html

[8] L. Breiman, “Random forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001. doi:10.1023/A:1010933404324

[9] B. P. Welford, “Note on a method for calculating corrected sums of squares and products,” Technometrics, vol. 4, no. 3, pp. 419–420, 1962. doi:10.1080/00401706.1962.10490022

[10] A. Correa Bahnsen, D. Aouada, A. Stojanovic and B. Ottersten, “Feature engineering strategies for credit card fraud detection,” Expert Systems with Applications, vol. 51, pp. 134–142, 2016. doi:10.1016/j.eswa.2015.12.030

[11] A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi and G. Bontempi, “Credit card fraud detection: a realistic modeling and a novel learning strategy,” IEEE Trans. Neural Networks and Learning Systems, vol. 29, no. 8, pp. 3784–3797, 2018. doi:10.1109/TNNLS.2017.2736643

[12] T. Saito and M. Rehmsmeier, “The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets,” PLOS ONE, vol. 10, no. 3, e0118432, 2015. doi:10.1371/journal.pone.0118432

[13] Apache Spark, “Structured Streaming Programming Guide.” https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html

[14] Delta Lake documentation. https://docs.delta.io/

[15] Streamlit documentation. https://docs.streamlit.io/

[16] Project repository: https://github.com/Imperator8000/real-time-fraud-detection